

**LAPORAN PRAKTIKUM SISTEM OPERASI
THREADS & CONCURRENCY**



**Dosen Pengampu :
Triawan Adi Cahyanto M.Kom**

**Disusun Oleh :
Subhan Maulana Andrianto 2410651014**

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025**

(PRAKTIKUM)

Topik 1: Membuat Thread Sederhana

Tujuan: Memahami cara membuat dan menjalankan thread menggunakan POSIX Threads (pthread) di Linux.

Penjelasan Konsep :

Thread adalah "pekerja kecil" dalam program Anda. Bayangkan Anda memiliki pabrik (program), dan thread adalah pekerja yang bisa mengerjakan tugas berbeda secara bersamaan.

```
GNU nano 7.2 thread_sederhana.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
// Fungsi yang akan dijalankan oleh thread
void* cetak_pesan(void* arg) {
    char* pesan = (char*) arg;

    for(int i = 1; i <= 5; i++) {
        printf("%s - hitungan ke-%d\n", pesan, i);
        sleep(1); // Tidur 1 detik
    }

    return NULL;
}

int main() {
    pthread_t thread1, thread2;
    printf("=== Program Dimulai ===\n\n");
    // Membuat 2 thread
    pthread_create(&thread1, NULL, cetak_pesan, "Thread 1");
    pthread_create(&thread2, NULL, cetak_pesan, "Thread 2");
    // Menunggu kedua thread selesai
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("\n=== Program Selesai ===\n");
    return 0;
}
```

```
root@DESKTOP-J9C60BQ:~# gcc -o thread_sederhana thread_sederhana.c
root@DESKTOP-J9C60BQ:~# ./thread_sederhana
=== Program Dimulai ===

Thread 1 - hitungan ke-1
Thread 2 - hitungan ke-1
Thread 1 - hitungan ke-2
Thread 2 - hitungan ke-2
Thread 2 - hitungan ke-3
Thread 1 - hitungan ke-3
Thread 1 - hitungan ke-4
Thread 2 - hitungan ke-4
Thread 1 - hitungan ke-5
Thread 2 - hitungan ke-5

=== Program Selesai ===
root@DESKTOP-J9C60BQ:~# |
```

Topik 2: Thread dengan Shared Memory

Tujuan: Memahami bagaimana thread berbagi memori dan masalah yang bisa terjadi.

Penjelasan Konsep :

Thread dalam satu proses berbagi memori yang sama. Ini seperti beberapa pekerja menggunakan papan tulis yang sama - mereka bisa saling menimpa tulisan.

```
root@DESKTOP-J9C60BQ: ~  
GNU nano 7.2 thread_shared.c  
#include <stdio.h>  
#include <pthread.h>  
int counter = 0; // Variabel yang dibagi bersama  
void* tambah_counter(void* arg) {  
    for(int i = 0; i < 100000; i++) {  
        counter++;  
    }  
    return NULL;  
}  
int main() {  
    pthread_t thread1, thread2;  
    printf("Counter awal: %d\n", counter);  
    // Buat 2 thread yang menambah counter  
    pthread_create(&thread1, NULL, tambah_counter, NULL);  
    pthread_create(&thread2, NULL, tambah_counter, NULL);  
    // Tunggu kedua thread selesai  
    pthread_join(thread1, NULL);  
    pthread_join(thread2, NULL);  
    printf("Counter akhir: %d\n", counter);  
    printf("Harusnya: 200000\n");  
    return 0;  
}  
  
root@DESKTOP-J9C60BQ:~# gcc -o thread_shared thread_shared.c  
root@DESKTOP-J9C60BQ:~# ./thread_shared  
Counter awal: 0  
Counter akhir: 104955  
Harusnya: 200000
```

Topik 3: Mengatasi Race Condition dengan Mutex

Tujuan: Belajar menggunakan mutex (mutual exclusion) untuk mengamankan shared data.

Penjelasan Konsep :

Mutex seperti "kunci pintu". Hanya satu thread yang boleh masuk ke area kritis (critical section) pada satu waktu.

```
root@DESKTOP-J9C60BQ: ~  
GNU nano 7.2 thread_mutex.c *  
#include <stdio.h>  
#include <pthread.h>  
int counter = 0;  
pthread_mutex_t lock; // Variabel mutex (kunci)  
  
void* tambah_counter(void* arg) {  
    for(int i = 0; i < 100000; i++) {  
        pthread_mutex_lock(&lock);  
        counter++;  
        pthread_mutex_unlock(&lock);  
    }  
    return NULL;  
}  
  
int main() {  
    pthread_t thread1, thread2;  
    // Inisialisasi mutex  
    pthread_mutex_init(&lock, NULL);  
  
    printf("Counter awal: %d\n", counter);  
  
    pthread_create(&thread1, NULL, tambah_counter, NULL);  
    pthread_create(&thread2, NULL, tambah_counter, NULL);  
  
    pthread_join(thread1, NULL);  
    pthread_join(thread2, NULL);  
  
    printf("Counter akhir: %d\n", counter);  
    printf("Harusnya: 200000\n");  
    // Hapus mutex  
    pthread_mutex_destroy(&lock);  
  
    return 0;  
}  
  
root@DESKTOP-J9C60BQ:~# nano thread_mutex.c  
root@DESKTOP-J9C60BQ:~# gcc -o thread_mutex thread_mutex.c  
root@DESKTOP-J9C60BQ:~# ./thread_mutex  
Counter awal: 0  
Counter akhir: 200000  
Harusnya: 200000  
root@DESKTOP-J9C60BQ:~#
```

Topik 4: Thread Pool Sederhana

Tujuan: Memahami konsep thread pool untuk efisiensi.

Penjelasan Konsep :

Daripada membuat thread baru setiap kali ada tugas, lebih baik punya "pool pekerja" yang siap mengerjakan tugas kapan saja.

```
root@DESKTOP-J9C60BQ: ~  
GNU nano 7.2 thread_pool.c  
#include <stdio.h>  
#include <pthread.h>  
#include <unistd.h>  
  
#define JUMLAH_THREAD 3  
#define JUMLAH_TUGAS 10  
  
void* kerjakan_tugas(void* arg) {  
    int id = *((int*) arg);  
  
    printf("Thread %d: Mulai mengerjakan tugas\n", id);  
    sleep(2); // Simulasi pekerjaan  
    printf("Thread %d: Selesai mengerjakan tugas\n", id);  
  
    return NULL;  
}  
  
int main() {  
    pthread_t threads[JUMLAH_THREAD];  
    int thread_ids[JUMLAH_THREAD];  
    printf("=== Thread Pool dengan %d threads ===\n\n", JUMLAH_THREAD);  
    // Buat pool of threads  
    for(int i = 0; i < JUMLAH_THREAD; i++) {  
        thread_ids[i] = i + 1;  
        pthread_create(&threads[i], NULL, kerjakan_tugas, &thread_ids[i]);  
    }  
  
    // Tunggu semua thread selesai  
    for(int i = 0; i < JUMLAH_THREAD; i++) {
```

```
        pthread_join(threads[i], NULL);  
    }  
  
    printf("\n=== Semua tugas selesai ===\n");  
    return 0;  
}
```

```
root@DESKTOP-J9C60BQ:~# gcc -o thread_pool thread_pool.c  
root@DESKTOP-J9C60BQ:~# ./thread_pool  
=== Thread Pool dengan 3 threads ===  
  
Thread 1: Mulai mengerjakan tugas  
Thread 2: Mulai mengerjakan tugas  
Thread 3: Mulai mengerjakan tugas  
Thread 1: Selesai mengerjakan tugas  
Thread 3: Selesai mengerjakan tugas  
Thread 2: Selesai mengerjakan tugas  
  
=== Semua tugas selesai ===
```

Topik 5: Data Parallelism

Tujuan: Memahami pemrosesan data paralel menggunakan multiple threads.

Penjelasan Konsep :

Membagi data besar menjadi bagian-bagian kecil dan memprosesnya secara paralel.

Langkah-langkah.

```
root@DESKTOP-J9C60BQ: ~ × + v
GNU nano 7.2 data_parallel.c *
#include <stdio.h>
#include <pthread.h>

#define UKURAN_ARRAY 1000
#define JUMLAH_THREAD 4

int array[UKURAN_ARRAY];
int jumlah_parsial[JUMLAH_THREAD];

typedef struct {
    int id;
    int mulai;
    int akhir;
} ThreadData;

void* hitung_jumlah(void* arg) {
    ThreadData* data = (ThreadData*) arg;
    int sum = 0;

    for(int i = data->mulai; i < data->akhir; i++) {
        sum += array[i];
    }

    jumlah_parsial[data->id] = sum;
    printf("Thread %d: Menghitung indeks %d-%d, hasil = %d\n",
           data->id, data->mulai, data->akhir-1, sum);
    return NULL;
}

int main() {
```

```

pthread_t threads[JUMLAH_THREAD];
ThreadData thread_data[JUMLAH_THREAD];

// Isi array dengan angka 1-1000
for(int i = 0; i < UKURAN_ARRAY; i++) {
    array[i] = i + 1;
}

printf("=== Data Parallelism: Menjumlahkan array ===\n\n");

int bagian = UKURAN_ARRAY / JUMLAH_THREAD;

// Buat thread untuk memproses bagian array
for(int i = 0; i < JUMLAH_THREAD; i++) {
    thread_data[i].id = i;
    thread_data[i].mulai = i * bagian;
    thread_data[i].akhir = (i + 1) * bagian;

    pthread_create(&threads[i], NULL, hitung_jumlah, &thread_data[i]);
}

// Tunggu semua thread selesai
for(int i = 0; i < JUMLAH_THREAD; i++) {
    pthread_join(threads[i], NULL);
}

// Jumlahkan hasil dari semua thread
int total = 0;
for(int i = 0; i < JUMLAH_THREAD; i++) {
    total += jumlah_parsial[i];
}

printf("\n=== Total keseluruhan: %d ===\n", total);
printf("Verifikasi: 1+2+...+1000 = %d\n", (UKURAN_ARRAY * (UKURAN_ARRAY + 1))
/ 2);

return 0;
}

```

```

root@DESKTOP-J9C60BQ:~# gcc -o data_parallel data_parallel.c
root@DESKTOP-J9C60BQ:~# ./data_parallel
=== Data Parallelism: Menjumlahkan array ===

Thread 0: Menghitung indeks 0-249, hasil = 31375
Thread 1: Menghitung indeks 250-499, hasil = 93875
Thread 2: Menghitung indeks 500-749, hasil = 156375
Thread 3: Menghitung indeks 750-999, hasil = 218875

=== Total keseluruhan: 500500 ===
Verifikasi: 1+2+...+1000 = 500500

```

Topik 6: Mendeteksi Jumlah CPU Core

Tujuan: Belajar mengoptimalkan program berdasarkan jumlah core processor.

```
GNU nano 7.2                                     cpu_cores.c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void* tugas_berat(void* arg) {
    int id = *((int*) arg);
    long hasil = 0;

    printf("Thread %d: Memulai komputasi...\n", id);

    // Simulasi komputasi berat
    for(long i = 0; i < 1000000000; i++) {
        hasil += i;
    }

    printf("Thread %d: Selesai!\n", id);
    return NULL;
}

int main() {
    // Deteksi jumlah CPU core
    int jumlah_core = sysconf(_SC_NPROCESSORS_ONLN);

    printf("=== Informasi Sistem ===\n");
    printf("Jumlah CPU core: %d\n\n", jumlah_core);

    pthread_t threads[jumlah_core];
    int thread_ids[jumlah_core];

    printf("Membuat %d thread (sesuai jumlah core)...\n\n", jumlah_core);

    // Buat thread sebanyak jumlah core
    for(int i = 0; i < jumlah_core; i++) {
        thread_ids[i] = i + 1;
        pthread_create(&threads[i], NULL, tugas_berat, &thread_ids[i]);
    }

    // Tunggu semua selesai
    for(int i = 0; i < jumlah_core; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\n=== Semua thread selesai ===\n");

    return 0;
}
```



```
root@DESKTOP-J9C60BQ:~# gcc -o cpu_cores cpu_cores.c
root@DESKTOP-J9C60BQ:~# ./cpu_cores
=== Informasi Sistem ===
Jumlah CPU core: 8

Membuat 8 thread (sesuai jumlah core)...

Thread 1: Memulai komputasi...
Thread 2: Memulai komputasi...
Thread 3: Memulai komputasi...
Thread 4: Memulai komputasi...
Thread 5: Memulai komputasi...
Thread 6: Memulai komputasi...
Thread 7: Memulai komputasi...
Thread 8: Memulai komputasi...
Thread 1: Selesai!
Thread 4: Selesai!
Thread 5: Selesai!
Thread 8: Selesai!
Thread 3: Selesai!
Thread 2: Selesai!
Thread 6: Selesai!
Thread 7: Selesai!

=== Semua thread selesai ===
```

(TUGAS)

1.KALKULATOR PARALEL

A. Buat file baru dengan cara ;

nano kalkulator_paralel.c

B. Masukan kodngan bawah kedalam file kalkulator.c

```
GNU nano 7.2 kalkulator_paralel.c
#include <stdio.h>
#include <pthread.h>

double a, b;
double hasil_tambah, hasil_kurang, hasil_kali, hasil_bagi;

void* tambah(void* arg) {
    hasil_tambah = a + b;
    return NULL;
}

void* kurang(void* arg) {
    hasil_kurang = a - b;
    return NULL;
}

void* kali(void* arg) {
    hasil_kali = a * b;
    return NULL;
}

void* bagi(void* arg) {
    if (b != 0) hasil_bagi = a / b;
    else hasil_bagi = 0;
    return NULL;
}

int main() {
    pthread_t t1, t2, t3, t4;
```

```

printf("Masukkan dua angka: ");
scanf("%lf %lf", &a, &b);

pthread_create(&t1, NULL, tambah, NULL);
pthread_create(&t2, NULL, kurang, NULL);
pthread_create(&t3, NULL, kali, NULL);
pthread_create(&t4, NULL, bagi, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_join(t3, NULL);
pthread_join(t4, NULL);

printf("\nHasil Penjumlahan: %.2lf", hasil_tambah);
printf("\nHasil Pengurangan: %.2lf", hasil_kurang);
printf("\nHasil Perkalian : %.2lf", hasil_kali);
printf("\nHasil Pembagian : %.2lf\n", hasil_bagi);

return 0;
}

```

Untuk menyimpan kodingan tekan ctrl + o lalu Enter Lalu tekan ctrl + x ctrl Untuk keluar kodingan.

C. Lalu Compile dengan cara memasukan kode di bawah kedalam terminal :

Gcc -o kalkulator_paralel kalkulator_paralel.c

D. Lalu Running dengan cara memasukan kode di bawah kedalam terminal :

./kalkulator_paralel

E. Masukan 2 angka lalu enter maka hasilnya akan seperti di bawah ini :

```

root@DESKTOP-J9C60BQ:~# gcc -o kalkulator_paralel kalkulator_paralel.c
root@DESKTOP-J9C60BQ:~# ./kalkulator_paralel
Masukkan dua angka: 2 7

Hasil Penjumlahan: 9.00
Hasil Pengurangan: -5.00
Hasil Perkalian : 14.00
Hasil Pembagian : 0.29

```

2.FILE PROCESSING

A. Buat file baru dengan cara ;

nano file_processing.c

B. Masukkan kodingan bawah kedalam file_processing.c

```
GNU nano 7.2 file_processing.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <time.h>

typedef struct {
    char* data;
    long start;
    long end;
    int count;
} ThreadData;

void* hitung_kata(void* arg) {
    ThreadData* td = (ThreadData*)arg;
    int in_word = 0;
    td->count = 0;
    for (long i = td->start; i < td->end; i++) {
        if (td->data[i] == ' ' || td->data[i] == '\n' || td->data[i] == '\t')
            in_word = 0;
        else if (in_word == 0) {
            in_word = 1;
            td->count++;
        }
    }
    return NULL;
}

int hitung_kata_single(char* data, long size) {
```

```

    int count = 0, in_word = 0;
    for (long i = 0; i < size; i++) {
        if (data[i] == ' ' || data[i] == '\n' || data[i] == '\t')
            in_word = 0;
        else if (in_word == 0) {
            in_word = 1;
            count++;
        }
    }
    return count;
}

int main() {
    FILE* f = fopen("input.txt", "r");
    if (!f) {
        printf("Gagal membuka file.\n");
        return 1;
    }
    fseek(f, 0, SEEK_END);
    long size = ftell(f);
    rewind(f);

    char* data = malloc(size + 1);
    fread(data, 1, size, f);
    data[size] = '\0';
    fclose(f);

    ThreadData td1 = {data, 0, size / 2, 0};
    ThreadData td2 = {data, size / 2, size, 0};
}

```

```

pthread_t t1, t2;

clock_t start = clock();
pthread_create(&t1, NULL, hitung_kata, &td1);
pthread_create(&t2, NULL, hitung_kata, &td2);
pthread_join(t1, NULL);
pthread_join(t2, NULL);
clock_t end = clock();
double waktu_multi = (double)(end - start) / CLOCKS_PER_SEC;
int total_multi = td1.count + td2.count;

start = clock();
int total_single = hitung_kata_single(data, size);
end = clock();
double waktu_single = (double)(end - start) / CLOCKS_PER_SEC;

printf("Jumlah kata (multi-thread): %d\n", total_multi);
printf("Jumlah kata (single-thread): %d\n", total_single);
printf("Waktu multi-thread : %.6f detik\n", waktu_multi);
printf("Waktu single-thread: %.6f detik\n", waktu_single);

free(data);
return 0;
}

```

Untuk menyimpan kodingan tekan ctrl + o lalu Enter Lalu tekan ctrl + x ctrl Untuk keluar kodingan.

C. Lalu Compile dengan cara memasukan kode di bawah kedalam terminal :

```
gcc -o file_processing file_processing.c
```

D. Lalu Running dengan cara memasukan kode di bawah kedalam terminal :

```
./file_processing
```

E. Dan ini hasil outputnya:

```

root@DESKTOP-J9C60BQ:~# gcc -o file_processing file_processing.c
root@DESKTOP-J9C60BQ:~# ./file_processing
Jumlah kata (multi-thread): 24
Jumlah kata (single-thread): 23
Waktu multi-thread : 0.000174 detik
Waktu single-thread: 0.000001 detik

```

3.PRODUCER CONSUMER

A. Buat file baru dengan cara ;

nano producer_consumer.c

B. Masukkan kodingan bawah kedalam file producer_consumer.c

```
GNU nano 7.2 producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10
#define TOTAL_ITEMS 20

int buffer[BUFFER_SIZE];
int count = 0, produced = 0, consumed = 0;
pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

void* producer(void* arg) {
    while (produced < TOTAL_ITEMS) {
        int item = rand() % 100;
        pthread_mutex_lock(&mutex);
        while (count == BUFFER_SIZE)
            pthread_cond_wait(&not_full, &mutex);
        buffer[count++] = item;
        produced++;
        printf("Producer menghasilkan: %d (jumlah buffer: %d)\n", item, count);
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
    return NULL;
}
```

```

void* consumer(void* arg) {
    while (consumed < TOTAL_ITEMS) {
        pthread_mutex_lock(&mutex);
        while (count == 0)
            pthread_cond_wait(&not_empty, &mutex);
        int item = buffer[--count];
        consumed++;
        printf("Consumer memproses : %d (sis buffer: %d)\n", item, count);
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);
        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);

    printf("\nSemua %d item telah diproduksi dan dikonsumsi.\n", TOTAL_ITEMS);
    return 0;
}

```

Untuk menyimpan kodingan tekan ctrl + o lalu Enter Lalu tekan ctrl + x ctrl Untuk keluar kodingan.

C. Lalu Compile dengan cara memasukan kode di bawah kedalam terminal :

```
gcc -o producer_consumer producer_consumer.c
```

D. Lalu Running dengan cara memasukan kode di bawah kedalam terminal :

```
./producer_consumer
```

E. Ini hasil outputnya :


```
root@DESKTOP-J9C60BQ:~# gcc -o producer_consumer producer_consumer.c
root@DESKTOP-J9C60BQ:~# ./producer_consumer
Producer menghasilkan: 83 (jumlah buffer: 1)
Consumer memproses : 83 (sis buffer: 0)
Producer menghasilkan: 86 (jumlah buffer: 1)
Consumer memproses : 86 (sis buffer: 0)
Producer menghasilkan: 77 (jumlah buffer: 1)
Producer menghasilkan: 15 (jumlah buffer: 2)
Consumer memproses : 15 (sis buffer: 1)
Producer menghasilkan: 93 (jumlah buffer: 2)
Producer menghasilkan: 35 (jumlah buffer: 3)
Consumer memproses : 35 (sis buffer: 2)
Producer menghasilkan: 86 (jumlah buffer: 3)
Producer menghasilkan: 92 (jumlah buffer: 4)
Consumer memproses : 92 (sis buffer: 3)
Producer menghasilkan: 49 (jumlah buffer: 4)
Producer menghasilkan: 21 (jumlah buffer: 5)
Consumer memproses : 21 (sis buffer: 4)
Producer menghasilkan: 62 (jumlah buffer: 5)
Producer menghasilkan: 27 (jumlah buffer: 6)
Consumer memproses : 27 (sis buffer: 5)
Producer menghasilkan: 90 (jumlah buffer: 6)
Producer menghasilkan: 59 (jumlah buffer: 7)
Consumer memproses : 59 (sis buffer: 6)
Producer menghasilkan: 63 (jumlah buffer: 7)
Producer menghasilkan: 26 (jumlah buffer: 8)
Consumer memproses : 26 (sis buffer: 7)
Producer menghasilkan: 40 (jumlah buffer: 8)
Producer menghasilkan: 26 (jumlah buffer: 9)
Consumer memproses : 26 (sis buffer: 8)
Producer menghasilkan: 72 (jumlah buffer: 9)
Producer menghasilkan: 36 (jumlah buffer: 10)
Consumer memproses : 36 (sis buffer: 9)
Consumer memproses : 72 (sis buffer: 8)
Consumer memproses : 40 (sis buffer: 7)
Consumer memproses : 63 (sis buffer: 6)
Consumer memproses : 90 (sis buffer: 5)
Consumer memproses : 62 (sis buffer: 4)
Consumer memproses : 49 (sis buffer: 3)
Consumer memproses : 86 (sis buffer: 2)
Consumer memproses : 93 (sis buffer: 1)
Consumer memproses : 77 (sis buffer: 0)
```

Semua 20 item telah diproduksi dan dikonsumsi.